МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»

Кафедра математической кибернетики и компьютерных наук

АНАЛИЗ СЛОЖНОСТИ БЫСТРОЙ И ПИРАМИДАЛЬНОЙ СОРТИРОВОК

ОТЧЁТ

студента 2 курса 211 группы
направления 02.03.02 — Фундаментальная информатика и информационные технологии
факультета компьютерных наук и информационных технологий
Ахметова Игоря Андреевича

Проверено:
доцент, к. ф.-м. н. _____ Ю. Н. Кондратова

Саратов 2026

СОДЕРЖАНИЕ

1 Реализация сортировок на основе сравнений . . . 3

1.1 Быстрая сортировка (Quick Sort) . . . 3

1.2 Пирамидальная сортировка (Heap Sort) . . . 4

2 Анализ сложности алгоритмов . . . 5

2.1 Анализ сложности быстрой сортировки . . . 5

2.1.1 Функция разделения `partition` . . . 5

2.1.2 Рекурсивная функция `quick_recursive` . . . 6

2.2 Анализ сложности пирамидальной сортировки . . . 7

2.2.1 Вспомогательная функция просеивания `heap_help` . . . 7

2.2.2 Основная функция `heap_sort` . . . 8

2.3 Сравнительный вывод . . . 10

1 Реализация сортировок на основе сравнений

1.1 Быстрая сортировка (Quick Sort)

Функция разделения partition:

```
int partition(vector<int>& mas, int low, int high) {
    uniform_int_distribution<int> distrib(low, high);
    int pivotIndex = distrib(rng); // случайный индекс опорного
    ↪ элемента
    swap(mas[pivotIndex], mas[high]);
    int pivot = mas[high];
    int i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
        if (mas[j] < pivot) {
            i++;
            swap(mas[i], mas[j]);
        }
    }
    swap(mas[i + 1], mas[high]);
    return (i + 1);
}
```

Рекурсивная функция quick_recursive:

```
void quick_recursive(vector<int>& mas, int low, int high) {
    if (low < high) {
        int pi = partition(mas, low, high);
        quick_recursive(mas, low, pi - 1);
        quick_recursive(mas, pi + 1, high);
    }
}
```

Основная функция quick_sort:

```
void quick_sort(vector<int>& mas, int size) {
    if (size > 0) quick_recursive(mas, 0, size - 1);
}
```

1.2 Пирамидальная сортировка (Heap Sort)

Вспомогательная функция просеивания heap_help:

```
void heap_help(vector<int>& mas, int n, int i) {
    int largest = i;  // предполагаем, что текущий узел - наибольший
    int left = 2 * i + 1;   // индекс левого потомка в куче
    int right = 2 * i + 2;  // индекс правого потомка
    if (left < n && mas[left] > mas[largest]) largest = left;
    if (right < n && mas[right] > mas[largest]) largest = right;
    // если наибольший - не текущий, меняем местами и продолжаем просеивать
    if (largest != i) {
        swap(mas[i], mas[largest]);
        heap_help(mas, n, largest);
    }
}
```

Основная функция heap_sort:

```
void heap_sort(vector<int>& mas, int size) {
    // построение кучи (перегруппировка массива)
    for (int i = size / 2 - 1; i >= 0; i--)
        heap_help(mas, size, i);
    // извлечение элементов из кучи по одному
    for (int i = size - 1; i > 0; i--) {
        swap(mas[0], mas[i]);
        heap_help(mas, i, 0); // просеиваем оставшуюся часть
    }
}
```

2 Анализ сложности алгоритмов

В данном разделе проводится анализ временной и пространственной сложности двух фундаментальных алгоритмов сортировки, основанных на сравнении элементов: быстрой сортировки (Quick Sort) и пирамидальной сортировки (Heap Sort). В отличие от линейных сортировок (подсчётом и поразрядной), данные алгоритмы не накладывают ограничений на диапазон значений элементов и работают корректно для любых типов данных, для которых определена операция сравнения. Однако их асимптотическая сложность в среднем или худшем случае ограничена снизу величиной $\Omega(n \log n)$.

2.1 Анализ сложности быстрой сортировки

Рассмотрим реализацию быстрой сортировки, представленную функциями `quick_sort`, `quick_recursive` и `partition`. Пусть n — количество элементов в сортируемом массиве. Алгоритм следует идее «разделяй и властвуй».

2.1.1 Функция разделения `partition`

Функция `partition` выполняет перестановку элементов в подмассиве `mas[low..high]` таким образом, что элементы, меньшие опорного (`pivot`), оказываются слева от него, а большие или равные — справа.

1. **Выбор опорного элемента:** В данной реализации используется рандомизированный выбор индекса опорного элемента:

   ```
   uniform_int_distribution<int> distrib(low, high);
   int pivotIndex = distrib(rng);
   swap(mas[pivotIndex], mas[high]);
   ```

   Это выполняется за время $O(1)$ и защищает алгоритм от деградации на отсортированных или частично отсортированных массивах.

2. **Цикл перестановки:** Основной цикл проходит по всем элементам подмассива от `low` до `high - 1`:

   ```
   for (int j = low; j <= high - 1; j++) {
       if (mas[j] < pivot) {
   ```

```
            i++;
            swap(mas[i], mas[j]);
        }
    }
```

Количество итераций цикла равно длине текущего подмассива $m = \text{high} - \text{low}$. На каждой итерации выполняется $O(1)$ операций сравнения и, возможно, обмена.

3. **Финальная установка опорного элемента:** После цикла опорный элемент перемещается на своё финальное место:

```
swap(mas[i + 1], mas[high]);
```

Это также $O(1)$.

Временная сложность `partition`**:** Прямо пропорциональна длине обрабатываемого подмассива m:

$$T_{\text{partition}}(m) = O(m).$$

2.1.2 Рекурсивная функция `quick_recursive`

Функция рекурсивно вызывает себя для левой и правой частей массива относительно опорного элемента:

```
int pi = partition(mas, low, high);
quick_recursive(mas, low, pi - 1);
quick_recursive(mas, pi + 1, high);
```

Анализ временной сложности зависит от того, насколько сбалансированно происходит разделение.

— **Лучший и средний случай:** Если опорный элемент делит массив на две примерно равные части, глубина рекурсии составляет $\log_2 n$. На каждом уровне рекурсии суммарно обрабатывается n элементов (во всех вызовах `partition` на данном уровне). Следовательно, временная сложность:

$$T_{\text{avg}}(n) = O(n \log n).$$

— **Худший случай:** Если опорный элемент постоянно оказывается минимальным или максимальным (что приводит к разделению на подмассивы размера 1 и $n-1$), глубина рекурсии становится равной n. Суммарное количество операций:

$$T_{\text{worst}}(n) = n + (n-1) + (n-2) + \cdots + 1 = \frac{n(n+1)}{2} = O(n^2).$$

Однако в моей реализации сортировки используется **рандомизированный выбор опорного элемента**, что делает худший случай крайне маловероятным на практике. Математическое ожидание времени работы для случайного выбора `pivot` составляет строго $O(n \log n)$ независимо от входных данных.

Общая временная сложность:

— Средний случай (и практически всегда): $O(n \log n)$.

— Худший случай (если считать без рандомизации): $O(n^2)$.

Анализ пространственной сложности: Алгоритм не требует выделения дополнительных массивов (работает *in-place*). Однако рекурсивные вызовы используют стек вызовов. Глубина стека в среднем случае составляет $O(\log n)$, а в худшем (без рандомизации) — $O(n)$. Таким образом, дополнительная память составляет:

— Средний случай: $O(\log n)$.

— Худший случай: $O(n)$.

2.2 Анализ сложности пирамидальной сортировки

Рассмотрим реализацию пирамидальной сортировки функции `heap_sort` и `heap_help`. Алгоритм базируется на структуре данных «двоичная куча».

2.2.1 Вспомогательная функция просеивания `heap_help`

Функция восстанавливает свойство невозрастающей пирамиды для поддерева с корнем в узле i, предполагая, что левое и правое поддеревья уже являются корректными кучами.

```
void heap_help(vector<int>& mas, int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
```

```
    int right = 2 * i + 2;
    if (left < n && mas[left] > mas[largest]) largest = left;
    if (right < n && mas[right] > mas[largest]) largest = right;
    if (largest != i) {
        swap(mas[i], mas[largest]);
        heap_help(mas, n, largest);
    }
}
```

Функция рекурсивно спускается по дереву, пока не восстановится свойство кучи. Максимальная глубина рекурсии равна высоте кучи, которая составляет $O(\log n)$, где n — размер кучи.

Временная сложность `heap_help`**:** Пропорциональна высоте кучи:

$$T_{\text{heap_help}}(n) = O(\log n).$$

2.2.2 Основная функция `heap_sort`

Функция `heap_sort` выполняет сортировку в два этапа.

Этап 1. Построение кучи (перегруппировка массива):

```
for (int i = size / 2 - 1; i >= 0; i--)
    heap_help(mas, size, i);
```

Цикл начинается с индекса последнего внутреннего узла ($\lfloor n/2 \rfloor - 1$) и вызывает `heap_help` для каждого узла, двигаясь к корню. На первый взгляд может показаться, что сложность этого этапа равна $O(n \log n)$, так как выполняется примерно $n/2$ вызовов функции сложности $O(\log n)$. Однако если подробнее рассмотреть примеры работы, то увидим, что большинство вызовов `heap_help` работает с поддеревьями небольшой высоты.

Суммарное количество операций при построении кучи можно оценить как:

$$\sum_{h=0}^{\lfloor \log_2 n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil \cdot O(h) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h} \right) = O(n).$$

Тогда **построение кучи выполняется за линейное время** $O(n)$.

Этап 2. Извлечение элементов из кучи по одному:

```
for (int i = size - 1; i > 0; i--) {
    swap(mas[0], mas[i]);
    heap_help(mas, i, 0);
}
```

На каждой из $(n-1)$ итераций максимальный элемент (корень кучи) перемещается в конец массива, после чего размер кучи уменьшается на 1, и вызывается `heap_help` для восстановления свойства кучи с корнем в индексе 0.

Сложность одной итерации составляет $O(\log i)$, где i — текущий размер кучи. Суммарная сложность этапа:

$$\sum_{i=1}^{n-1} O(\log i) = O(n \log n).$$

Общая временная сложность: Суммируя затраты на обоих этапах, получаем:

$$T(n) = O(n) + O(n \log n) = O(n \log n).$$

Важно отметить, что в отличие от быстрой сортировки, пирамидальная сортировка гарантирует $O(n \log n)$ в **любом** случае (худшем, среднем, лучшем).

Анализ пространственной сложности: Алгоритм работает *in-place*, не требуя выделения дополнительных массивов. Рекурсивные вызовы `heap_help` используют стек глубиной $O(\log n)$. Таким образом, дополнительная память составляет:

$$M(n) = O(\log n) \text{ (на стек вызовов).}$$

2.3 Сравнительный вывод

Алгоритм	**Время (среднее)**	**Время (худшее)**	**Память**
Быстрая сортировка (Quick Sort)	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Пирамидальная сортировка (Heap Sort)	$O(n \log n)$	$O(n \log n)$	$O(\log n)$

Таблица 2.1 – Сравнение характеристик сортировок на основе сравнений

— **Быстрая сортировка** является алгоритмом выбора в большинстве стандартных библиотек (например, `std::sort` в C++) благодаря отличной производительности на практике и хорошей локальности данных. Рандомизированный выбор опорного элемента решает проблему худшего случая на отсортированных массивах. Однако в системах реального времени, где важна гарантированная производительность, может быть предпочтительнее другой алгоритм.

— **Пирамидальная сортировка** гарантирует $O(n \log n)$ в худшем случае. Она предпочтительна в системах с жёсткими ограничениями по памяти и когда необходимо гарантированное время выполнения. Также пирамидальная сортировка является основой для реализации очередей с приоритетом.